Low-Latency MJPEG Streaming on ESP32-S3-EYE

Measuring Resolution Trade-offs Under Constrained Network Conditions

Matt Christiansen

CSCE 5612 Embedded Systems
University of North Texas

Spring 2026

Demo

- **LINK:** `https://youtu.be/z7-dr4xzfsY`

Project Framing

Core idea

This project uses a constrained embedded camera platform as a **measurement testbed** to answer a systems question:

Which stream resolution best preserves low latency and stream stability when both hardware and bandwidth are limited?
This is a Multi-Objective Optimization Problem (MOP)

- Platform: ESP32-S3-EYE with OV2640 camera
- Stream format: HTTP MJPEG over Wi-Fi
- Analysis focus: latency, effective bitrate, frame continuity, and stability
- Motivation: real-time remote viewing in constrained environments

ESP32-S3-EYE

What Makes This an Embedded Systems Project

- The ESP32-S3-EYE defines the real performance boundary:
  - constrained CPU
  - constrained memory / PSRAM
  - simple MJPEG pipeline instead of H.264
  - practical Wi-Fi limitations on a low-cost device
- The firmware must remain stable while repeatedly capturing, encoding, and transmitting frames.
- The project studies how **embedded constraints interact with network constraints**.

So what? Why This Problem Matters...

- In teleoperation decisions depend on live video.
- Embedded platforms do not have the compute budget of desktop or edge GPU systems.
- Wi-Fi links can become the bottleneck before application logic does.
- Higher resolution improves clarity, but it also increases:
  - encoding cost
  - network load
  - queueing delay
  - risk of frame stalls or drops

The goal is not "highest quality."
The goal is: what actually works under constraints?

Research Question and Objectives

Research question

Given an ESP32-S3-EYE streaming MJPEG frames over Wi-Fi, which resolution provides the best trade-off between latency, bandwidth consumption, and stream robustness under constrained network conditions?

Objectives

1. Build a working embedded MJPEG streaming system on the ESP32-S3-EYE
2. Instrument the stream so each frame can be measured at the client
3. Run controlled experiments across multiple resolutions and bandwidth ceilings
4. Produce evidence-based guidelines for choosing a usable stream configuration

System Architecture

OV2640 Camera → ESP32-S3-EYE Firmware → Wi-Fi / HTTP MJPEG → Python Logger

On-device responsibilities

- capture frame
- configure resolution / JPEG settings
- package frame metadata
- stream over HTTP

Off-device responsibilities

- receive Motion JPEG (MJPEG) stream
- parse frames and timestamps
- compute latency / bitrate / FPS
- save CSV logs and plots

The embedded system generates the data.
The measurement pipeline extracts the insight.

RTOS Architecture (ESP32-S3)

Key Idea: Convert a sequential firmware design into a concurrent real-time pipeline.

Task Decomposition:

- **Core 0 — Network / Adaptation Task**
  - Manages Wi-Fi and stream delivery
  - Monitors system state (RSSI, memory, latency)
  - Decides when to adjust camera parameters
- **Core 1 — Camera Task**
  - Handles frame capture and sensor configuration
  - Maintains continuous streaming pipeline

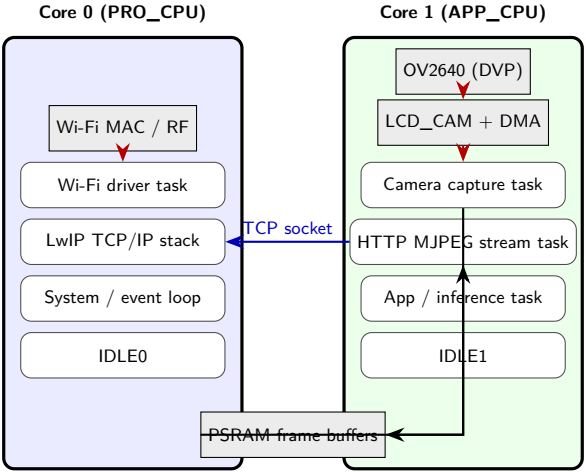
Inter-Task Communication:

- RTOS queues for commands/events
- Mutex-protected shared status

Why This Matters:

- Eliminates blocking between camera and network
- Enables real-time adaptation under constraints
- Reflects real embedded streaming systems

ESP32-S3 Task Pinning: Core 0 vs Core 1



- - DMA path → Cross-core data **xTaskCreatePinnedToCore** isolates the DMA consumer from Wi-Fi latency. The camera-side work is running on core 1, and the network/status work is running on core 0

Inside the ESP32-S3-EYE Image Pipeline (1)

What happens after the OV2640 captures an image?

1. **Optical capture at the OV2640 sensor**
   - The OV2640 camera sensor captures light through the lens and converts it into pixel data.
   - Internal sensor logic applies the configured resolution, timing, and JPEG compression settings.
2. **Parallel camera transfer into the ESP32-S3**
   - The sensor sends image data over the camera interface using pixel clock, synchronization signals, and parallel data lines.
   - The ESP32-S3 receives this stream and assembles it into a frame buffer.
3. **Frame buffer placement in memory**
   - Captured JPEG frames are stored in on-device memory as camera frame buffers.
   - In this project, limited buffering is important because memory constraints directly affect stability and frame handling.

Inside the ESP32-S3-EYE Image Pipeline (2)

1. **Firmware access to the captured frame**
   - The firmware retrieves a completed frame from the camera driver using the ESP32 camera API.
   - Metadata such as frame size, byte length, and timing information can be attached before transmission.
2. **MJPEG streaming over HTTP**
   - Each JPEG image is sent as one part of a multipart HTTP MJPEG stream.
   - The device repeatedly captures, buffers, and transmits frames to sustain live video delivery.

Sensor capture → frame buffering → firmware handling → MJPEG transmission

ESP32-S3-EYE

Firmware Support for Measurement

- **Platform Configuration**
  - Configured for `CAMERA_MODEL_ESP32S3_EYE`
  - Uses ESP32-S3-EYE camera pin mapping and PSRAM support
- **HTTP Camera Server**
  - Exposes `/stream`, `/capture`, `/status`, and `/control` endpoints
  - Streams MJPEG frames over HTTP multipart response
- **Experimental Instrumentation**
  - Added per-frame metadata headers:
    - `X-Frame-Id`
    - `X-Capture-Time-Us`
    - `X-Timestamp`
  - Enabled frame-level tracking from camera capture to downstream logging
- **Runtime Control**
  - Supports changing frame size and quality through HTTP control interface
  - Allowed repeated testing across multiple resolution settings

Network Path and Traffic Control Setup

- The ESP32-S3-EYE streams Motion JPEG (MJPEG) frames over **802.11 Wi-Fi** to the local router
- The router forwards the stream through the local network to a **Raspberry Pi**
- The Raspberry Pi runs a proxy on **port 8080**
- Linux `tc` is applied on the Raspberry Pi network interface to impose bandwidth ceilings

Effective data path

`ESP32-S3-EYE` → `802.11 Wi-Fi` → `Router` → `Raspberry Pi:8080 Proxy` → `tc-limited link` → `Python logger`

- This allows the experiment to simulate constrained network conditions such as:
  - 2 Mbps
  - 5 Mbps
  - 10 Mbps
  - 20 Mbps
- The Raspberry Pi acts as the control point for repeatable bandwidth shaping

Traffic Control (tc)

- **Purpose:** Emulate constrained network conditions
- **Mechanism:** Acts as a bottleneck
- **Shaping:**
  - Limits bandwidth (e.g., 20, 10, 5, 2 Mbps)
  - Smooths bursts → more realistic network behavior
- **Command:**
  `sudo tc qdisc replace dev wlan0 root tbf rate xxmbit burst 32kb latency 50ms`

Key Insight: `tc` enables controlled bandwidth constraints, allowing systematic evaluation of latency under degraded network conditions.

Link: `https://man7.org/linux/man-pages/man8/tc.8.html`

Breaking Down `tc` Command

```
sudo tc qdisc replace dev wlan0 root tbf rate 5mbit burst 32kb latency 50ms
```

- `sudo` — Run with administrative privileges
- `tc` — Traffic control utility
- `qdisc` — Queueing discipline (controls packet scheduling)
- `replace` — Apply or update the rule
- `dev wlan0` — Target network interface (WiFi 802.11)
- `root` — Apply at the root of the interface
- `tbf` — Token Bucket Filter (rate-limiting algorithm)
- `rate 5mbit` — Maximum bandwidth allowed
- `burst 32kb` — Allowed burst size before throttling
- `latency 50ms` — Maximum buffering delay

Key Insight: This command enforces a strict bandwidth cap with controlled buffering, simulating real-world network constraints for latency analysis.

Why the ESP32-S3-EYE Fits This Study

- The ESP32-S3-EYE provides exactly the components needed for a constrained streaming experiment:
  - a 2 MP camera
  - an ESP32-S3 MCU
  - integrated 2.4 GHz Wi-Fi
  - 8 MB PSRAM and 8 MB flash
- The ESP32-S3 itself includes:
  - dual-core Xtensa LX7 CPU up to 240 MHz
  - a digital camera interface
  - Wi-Fi 802.11 b/g/n support
- For this project, the important path is:

$$\text{Camera} \rightarrow \text{ESP32-S3} \rightarrow \text{MJPEG stream} \rightarrow \text{Wi-Fi}$$

- This makes the board a useful **low-cost, resource-constrained testbed** for studying latency and stream stability (Type 2 constrained device).

OV2640 Camera: Hardware Constraints

- **Sensor Overview:**
  - 2 MP CMOS image sensor (UXGA: 1600×1200)
  - Integrated image processing and JPEG compression
- **Frame Rate vs Resolution:**
  - around 15 fps at full resolution (UXGA)
  - around 30 fps at lower resolutions (e.g., VGA)
  - Higher resolution → lower achievable frame rate
- **Latency Implications:**
  - Each frame must be:
    - captured
    - processed (JPEG compression)
    - transmitted over Wi-Fi
  - Higher resolution increases:
    - processing time
    - frame size
    - transmission delay
- **Key Constraint:**
  - The camera itself limits throughput and timing
  - **Resolution directly impacts latency and stability**

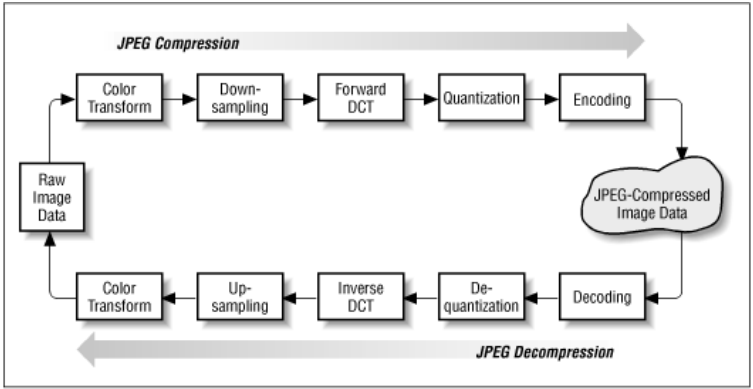
MJPEG and Its Impact on This Study

- MJPEG encodes each frame independently:

$$\text{Frame}_1, \text{Frame}_2, \text{Frame}_3, \ldots$$

- **Advantages:**
  - **Low complexity:** No inter-frame dependencies $\rightarrow$ low encoding latency
  - **Error Tolerance:** MJPEG outperforms inter-frame CODECs in noisy environments (like mine)
  - Simple pipeline $\rightarrow$ suitable for ESP32
- **Disadvantages:**
  - Larger frame sizes $\rightarrow$ higher bandwidth usage
  - More sensitive to network congestion
- **Implication for this project:**
  - Network becomes the dominant bottleneck
  - Resolution directly impacts latency and stability

JPEG Compression Pipeline



DCT: Converts spatial pixel data into frequency components (high freq = rapid shift of pixel values).
Quantization: Reduces precision of high-frequency components (lossy step, major compression source).
Encoding: Applies entropy coding (e.g., Huffman) to efficiently store remaining data.
Key Insight: Most compression comes from quantization, which trades image quality for reduced data size and faster transmission.

Luminance vs Chrominance (Why JPEG Works)

Key Idea: Human vision is more sensitive to brightness than color.

- Images are often represented as:
  - **Luminance (Y)** — brightness / intensity
  - **Chrominance (Cb, Cr)** — color information
- The human eye:
  - is highly sensitive to luminance (edges, structure)
  - is less sensitive to chrominance (color detail)
- JPEG exploits this:
  - preserves luminance detail
  - aggressively compresses chrominance

Implication for this project:

- lower resolutions can still look acceptable
- visual quality depends more on structure than raw pixel count
- supports why 320×240 remains usable under constraints

Compression removes color detail first, not structure.

Experimental Variables

Independent variables

- Resolution profiles:
  - P1: 160×120
  - P2: 320×240
  - P3: 480×320
- Network ceilings using Linux `tc`
  - example ceilings: 20 Mbps, 10 Mbps, 5 Mbps, 2 Mbps
- Repeated runs per configuration

Controlled / fixed choices

- ESP32-S3-EYE hardware platform
- MJPEG over HTTP transport
- same client-side logger and analysis scripts
- same environment and collection workflow per trial

Resolution is treated as a system-level control knob, not just a visual quality setting.

Measurement Pipeline

Per-frame data recorded

- frame ID
- embedded send timestamp
- client receive timestamp
- frame size in bytes
- active profile / network condition label

Derived metrics

- one-way stream delivery latency estimate
- effective bitrate
- effective frame rate
- frame drops and stalls
- latency distributions

Experiments

I collected over 300 controlled experimental runs across multiple resolutions and bandwidth constraints, allowing statistically meaningful comparison of streaming performance.

Evaluation Methodology

1. Select a resolution profile on the ESP32-S3-EYE
2. Apply a network ceiling using Linux traffic control
3. Stream MJPEG frames to a Python client
4. Log per-frame measurements to CSV
5. Repeat trials for each profile / condition pair
6. Aggregate results into comparative plots and summary statistics

Why this design

This is a **repeatable measurement pipeline**, not a one-off demonstration. The repeated runs make it possible to compare profiles statistically instead of relying on a single visual impression.

Understanding p95 and p99 Latency

What do p95 and p99 mean?

- **p95 latency**: 95% of frames arrive faster than this value
- **p99 latency**: 99% of frames arrive faster than this value

Why not just use the average?

- Average latency hides spikes and stalls
- A stream can have a good average but still freeze occasionally

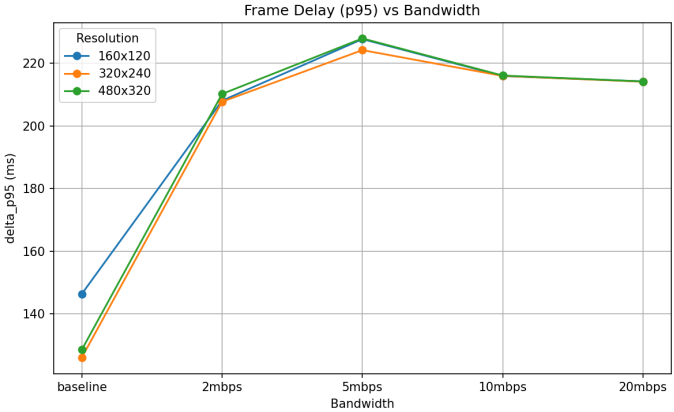
Why this matters for streaming

- Real-time systems depend on **consistency**, not just speed
- High p95 / p99 values indicate:
  - frame delays
  - buffering
  - unstable viewing experience

Let's get to

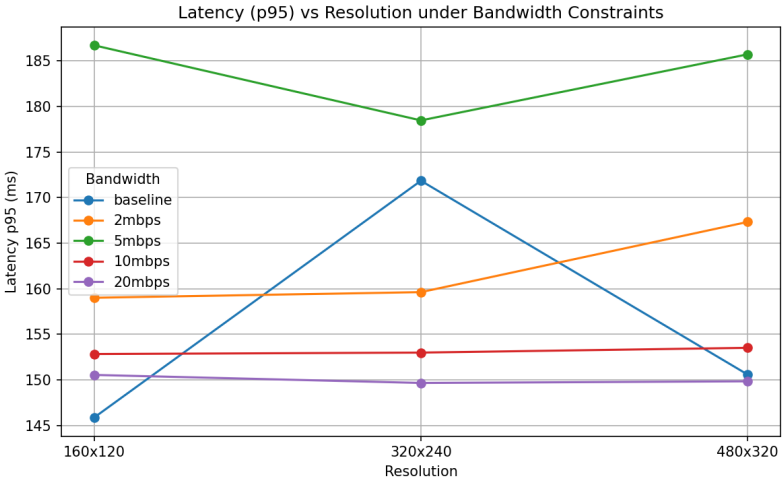
Results.

Frame Delay (p95) vs Bandwidth



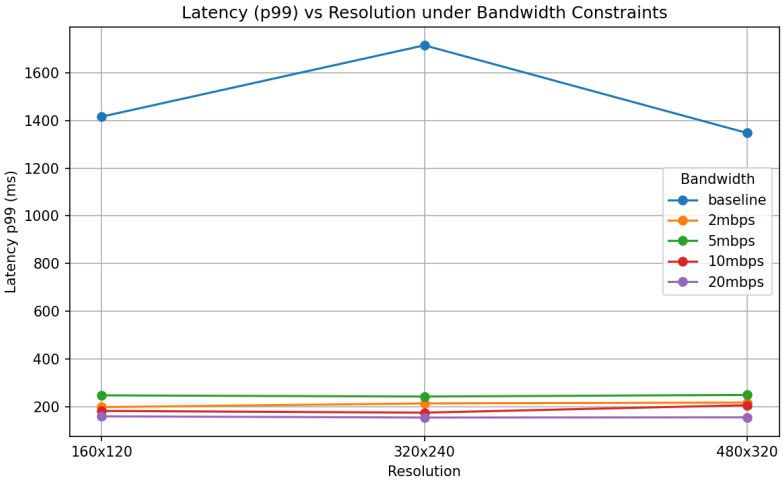
Key Insight: Bandwidth is the dominant factor in stream stability. Under constrained conditions, all resolutions experience increased frame delay, with only minor differences between them.

Latency (p95) vs Resolution



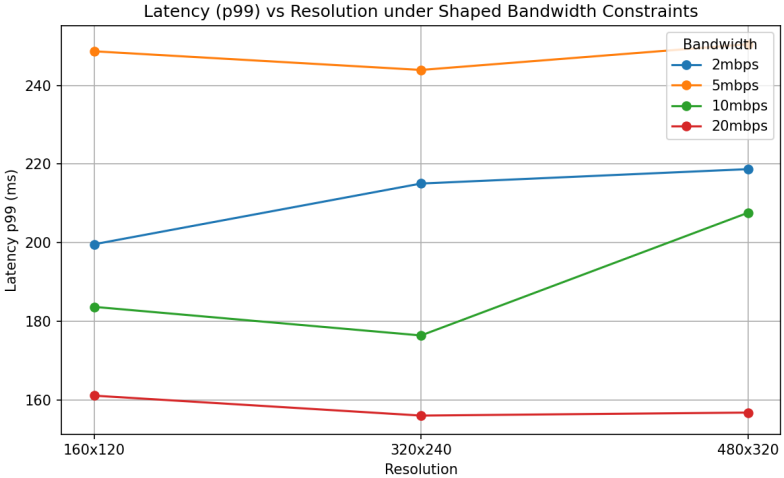
Key Insight: 320×240 achieves the lowest latency under constrained bandwidth.

Latency (p99) vs Resolution



Key Insight: 320×240 achieves the lowest latency under constrained bandwidth.

Latency (p99) vs Resolution without Baseline



Key Insight: 320×240 achieves the lowest latency under constrained bandwidth.

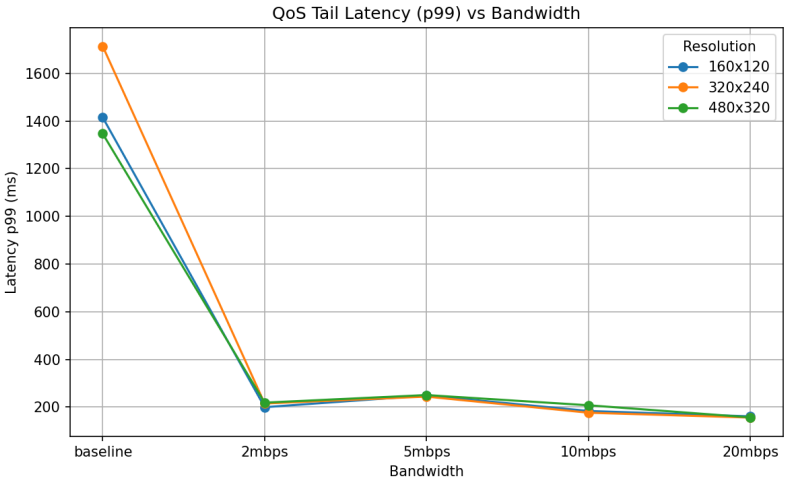
Latency (p99) vs Resolution under Shaped Bandwidth

- Removing the baseline isolates behavior under stable, QoS-controlled conditions
- 320×240 consistently achieves the lowest or near-lowest tail latency across bandwidths
- Higher resolution (480×320) increases latency due to larger frame sizes and transmission cost
- Lower resolution (160×120) provides minimal latency benefit beyond a certain point

Key Insight

Under constrained but stable network conditions, latency is driven more by frame size and system throughput than raw bandwidth.

QoS Graph



- Bandwidth shaping reduces tail latency by preventing queue buildup, demonstrating a core QoS principle: controlled traffic flow improves service consistency more than raw bandwidth.

Unexpected Finding: Baseline Instability

Observation

Unconstrained (baseline) streaming produced the highest tail latency (p99), significantly worse than constrained bandwidth conditions.

What was expected

- Removing bandwidth limits should reduce latency

What actually happened

- Baseline shows extreme p99 latency (1000+ ms spikes)
- Constrained bandwidth (2–20 Mbps) remains below 250 ms

Explanation

- No traffic shaping $\rightarrow$ burst transmission
- Frames accumulate in buffers
- Older frames are delivered late
- Wi-Fi variability amplifies delays

Controlled bandwidth improves consistency by preventing queue buildup.

Key Finding

320×240 provides the best trade-off between latency, stability, and visual quality under constrained bandwidth.

- Under constrained bandwidth, 320×240 represents a **Pareto-optimal** point, balancing latency, stability, and visual fidelity. Higher resolutions introduce additional delay and instability, while lower resolutions provide minimal latency benefit.
- Higher resolution (480×320):
  - increases latency
  - increases instability
- Lower resolution (160×120):
  - does not significantly reduce latency
  - reduces visual usefulness

A Note on Visual Quality!!

Key Observation: Video quality is not purely objective.

- Traditional evaluation relies on objective metrics:
  - latency
  - frame rate (FPS)
  - bitrate
- However, research in video codecs shows that **perceived visual quality is inherently subjective**
  - human perception varies across users
  - small artifacts may be acceptable depending on context
  - usefulness depends on the task (e.g., monitoring vs fine detail)

Implication for this work:

- 320×240 is optimal based on measurable system performance
- but **true "best quality"** depends on human evaluation
- a complete study would include user-based perceptual testing (the double stimulus continous quality scale (DSCQS) method)

Objective metrics guide system design. Human perception defines usefulness.

Connecting to Quality of Service (QoS)

Key Insight: This is fundamentally a QoS problem.

Observed in this study:

- Latency distributions (p95, p99) capture **tail behavior**, not just averages
- Frame stalls and spikes indicate **queueing delay and instability**
- Bandwidth shaping (`tc`) directly controls **traffic flow and buffering**

QoS Interpretation:

- Unconstrained traffic $\rightarrow$ burst transmission $\rightarrow$ buffer buildup $\rightarrow$ high tail latency
- Controlled bandwidth $\rightarrow$ smoother flow $\rightarrow$ reduced queueing $\rightarrow$ improved consistency

Core Takeaway

Bandwidth shaping improves tail latency by controlling queueing delay — a fundamental QoS principle.

Implication:

- Embedded video streaming must be designed with **network QoS constraints in mind**, not just encoding or resolution choices

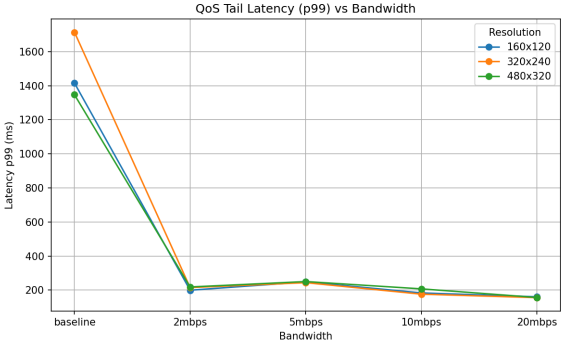
QoS Tail Latency Under Bandwidth Constraints

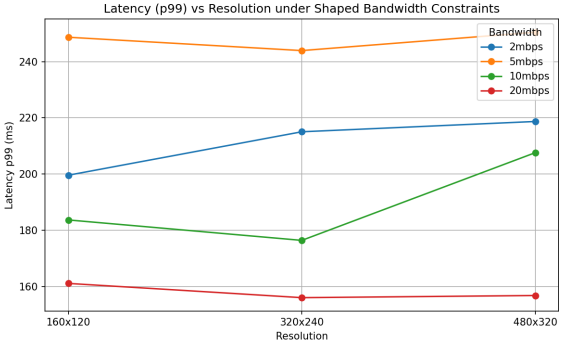
- In real-time streaming, QoS depends not only on average latency, but on **tail latency** and consistency
- Here, **p99 latency** acts as a QoS indicator of rare but important delay spikes
- Large p99 spikes suggest **queue buildup, burstiness, and unstable delivery**
- Bandwidth shaping with `tc` improves QoS by limiting burst transmission and reducing queueing delay

Interpretation

This study shows that controlling bandwidth can improve QoS consistency, even when raw bandwidth is reduced.

From Unstable Network to Optimal Configuration





Step 1: Control the Network (QoS)

Unshaped traffic → bursty delivery → high tail latency

Bandwidth shaping → bounded latency → stable system

Step 2: Optimize Within Constraints

320×240 consistently minimizes tail latency

Higher resolution → increased transmission cost

Lower resolution → diminishing returns

Core Insight

First enforce QoS to stabilize the system, then optimize resolution within those constraints.

Counterintuitive Findings

- It makese logical sense that 160x120 would have the least p95 and p99 latency on all ceilings.
- It only did so at the 2Mbit ceiling, once bandwidth increased, the larger frames likely aligned better with the newtowkrs packet flow because of TCP behavior (Nagle), Wi-Fi contention, variance (queueing theory), retransmission impact
- fewer instances where a single video frame needed to be split into multiple packets
- Bandwidth shaping actually makes latency **better**? Through ORDERLY Delivery and predictable flow and avoid backlog

Counterintuitive Reasonings

- Latency is not just about size — it's about how *smoothly* data flows through the system.

Mechanism 1: Small Frames Can Stall (TCP behavior)

- 160×120 frames are small
- TCP tries to batch small data (Nagle's algorithm:the "small-packet problem") `https://en.wikipedia.org/wiki/Nagle%27s_algorithm`
- Receiver delays ACKs

Mechanism 2: Wi-Fi hates lots of tiny transmissions

- Every transmission requires: CSMA/CA, Backoff, ACK
- 160x120 (more frames per second.. more "fights" for the channel
- 320x240 (fewer, larger transmission, more efficient use of airtime)

Mechanism 3:Variance matters more than average

- P99 is driven by variance **not** mean
- 160x120 frame sizes fluctuate wildly
- 320x240 more stable relative size

Queueing systems don't like variability

Mechanism 4 : Loss hurts small frames more

- small frames fail harder
- EXAMPLE: 160X120- 3 PACKETS Per frame, lose 1 (33% of frame lost)
- EXAMPLE: 320x240 - 11 PACKETS Per frame, lose 1 (only 9% impact!)

Queueing systems don't like variability

Latency Explained

$$\text{latency} \approx \underbrace{\frac{b_f}{R}}_{\text{transmission}} + \underbrace{W_q}_{\text{queueing}} + \underbrace{\delta_{\text{Nagle}}}_{\text{TCP stall}} + \underbrace{\delta_{\text{MAC}}}_{\text{802.11 overhead}} \tag{1}$$

TRUE BOTTLENECK

- system coordination **across** layers
- Furthermore, Wi-Fi in the 2.4 GHz band (used by the ESP32-S3) operates in a contention-based medium.
- Each TXOP acquisition is a probabilistic event.
- Small-frame streams require more TXOPs per unit time (more contention events), increasing collision probability and retransmission rate. Larger frames (320×240) reduce TXOP frequency, lowering contention-induced delay variance — which directly reduces P99 tail latency.
- P99 latency in a streaming system is not primarily determined by transmission time but by queue depth at the bottleneck

Deliverables

- Working ESP32-S3-EYE firmware for HTTP MJPEG streaming
- Configurable resolution profiles for controlled experiments
- Python-based measurement client and CSV logging pipeline
- Plots comparing latency, bitrate, FPS, and drops across conditions
- Design guidance for selecting a practical stream resolution on constrained hardware

Main contribution

A concrete, data-driven answer to a real engineering question: **what stream configuration keeps video usable when both hardware and bandwidth are constrained?**

Challenges

- **System Orchestration**
  - Coordinating multiple components across devices
  - ESP32 firmware, WiFi network, Raspberry Pi proxy, and logging pipeline
- **Networking Complexity**
  - Unpredictable WiFi behavior (contention, variability)
  - Ensuring consistent and controlled bandwidth using `tc`
- **Streaming Pipeline**
  - Maintaining stable MJPEG stream under constrained conditions
  - Handling dropped frames and inconsistent timing
- **Data Collection**
  - Capturing accurate per-frame timestamps
  - Synchronizing capture, transmission, and logging

Key Insight: The primary challenge was not individual components, but integrating them into a reliable end-to-end measurement system.

Limitations and Scope

Current scope

- MJPEG only
- ESP32-S3-EYE only
- Wi-Fi-based streaming
- measurement-focused study

Not yet included

- H.264 or hardware video codecs
- adaptive bitrate / dynamic profile switching
- 5G or cellular network testing
- closed-loop teleoperation experiments

These limits are acceptable because the project is intended to establish a constrained embedded baseline.

Limitations: What about PSNR?

- **PSNR (Peak Signal-to-Noise Ratio)**
  - widely used objective metric for image/video quality
  - measures **pixel-level fidelity**:
    - how closely the reconstructed image matches the original at the pixel level
- **Limitation:**
  - PSNR measures mathematical similarity, not human perception
  - does not capture:
    - perceptual artifacts
    - usability in real-time systems
- **Implication for this work:**
  - this study focuses on **latency, stability, and system performance**
  - higher PSNR (higher resolution) did not lead to better overall system behavior

PSNR captures fidelity. This work evaluates system-level usefulness.

Why This Project Matters for Future Work (For my research)

- Establishes a baseline before moving to more advanced codecs and network settings (*Tail Latency Decomposition in Embedded MJPEG Streaming: A Systems Analysis of Queueing, Transport, and MAC Dynamics on the ESP32-S3*)?
- Helps identify which variables matter most before adding system complexity
- Connects directly to future work in:
  - teleoperation video pipelines
  - adaptive streaming
  - cellular / 5G transport (URLLC)
  - edge and robotics systems

Research trajectory

The device is the medium. The long-term value comes from the measurement framework and the design rules it produces.

Conclusion

- This project asks a focused systems question using a real embedded platform.
- The ESP32-S3-EYE serves as a low-capability, high-constraint baseline.
- The measurement pipeline turns streaming behavior into quantitative evidence.
- The expected outcome is a practical resolution-selection guideline for constrained video streaming.

The goal was to understand the system.

References

Vladimir Vechtomov.
Low Latency H.264 Encoding for Teleoperation.
Master's thesis, Tampere University, 2023.

Miran Seo, Samraat Gupta, and Youngjib Ham.
Exploratory study on time-delayed excavator teleoperation in virtual lunar construction simulation: Task performance and operator behavior.
Automation in Construction, 168:105871, 2024.

Imran Khan, Tuyen X. Tran, Matti Hiltunen, Theodore Karagioules, and Dimitrios Koutsonikolas.
An Experimental Study of Low-Latency Video Streaming over 5G.
IEEE MeditCom, 2024.

Jim Gettys and Kathleen Nichols.
Bufferbloat: Dark buffers in the internet.
ACM Queue, 9(11):40–54, 2011.
https://dl.acm.org/doi/10.1145/2063166.2071893

References

Kathleen Nichols and Van Jacobson.
Controlling queue delay.
ACM Queue, 10(5):20–34, May 2012.
https://queue.acm.org/detail.cfm?id=2209336

Neal Cardwell, Yuchung Cheng, C. Stephen Gunn, Soheil Hassas Yeganeh, and Van Jacobson.
BBR: Congestion-based congestion control.
ACM Queue, 14(5):20–53, 2016.
https://queue.acm.org/detail.cfm?id=3022184

Jeffrey Dean and Luiz André Barroso.
The tail at scale.
Communications of the ACM, 56(2):74–80, February 2013.
https://cacm.acm.org/research/the-tail-at-scale/

John Nagle.
Congestion control in IP/TCP internetworks.
RFC 896, Internet Engineering Task Force, January 1984.
https://www.rfc-editor.org/rfc/rfc896

Robert Braden (Ed.).
Requirements for internet hosts – communication layers.
RFC 1122, Internet Engineering Task Force, October 1989.
https://www.rfc-editor.org/rfc/rfc1122

Jungwon Lee and Sunghyun Hwang.
A-MPDU aggregation with optimal number of MPDUs for delay requirements in IEEE 802.11ac.
PLOS ONE, 14(3):e0213888, March 2019.
https://doi.org/10.1371/journal.pone.0213888

Hulya Seferoglu, Konstantinos Oikonomou, and Ioannis Stavrakakis.
Performance study of frame aggregation mechanisms in the new generation WiFi.

In *CEUR Workshop Proceedings*, volume 1689, 2015.
https://ceur-ws.org/Vol-1689/paper7.pdf

Tim Brecht, Chenxi Liu, and Jochen Hecht.
Demystifying frame aggregation in 802.11 networks.
Technical report, University of Waterloo, 2021.
https://cs.uwaterloo.ca/~brecht/papers/demyst-comp-comm-2021.pdf

Erfan Sharafzadeh, Soudeh Ghorbani, and others.
Understanding the impact of host networking elements on traffic bursts.
In *Proceedings of USENIX NSDI*, 2023.
https://www.usenix.org/conference/nsdi23/presentation/sharafzadeh